

## Laboratorio #5 ALE

### Introducción

El objetivo del laboratorio fue desarrollar la infraestructura necesaria para que un agente interactúe con un juego de Atari 2600 mediante Gymnasium y Arcade Learning Environment (ALE). No se entrenó un modelo: se implementaron agentes de referencia y funciones reutilizables para crear entornos, ejecutar episodios, acumular métricas y grabar las partidas. Como caso de estudio se utilizó ALE/SpaceInvaders-v5.

### Arcade Learning Environment

ALE es una plataforma diseñada para investigar y evaluar agentes generales en juegos de Atari 2600. Ofrece una interfaz uniforme de aprendizaje por refuerzo: el agente recibe una observación, selecciona una acción y el entorno devuelve la nueva observación, una recompensa y señales de finalización. De esta manera no es necesario implementar una integración diferente para cada juego y es posible comparar algoritmos bajo condiciones reproducibles [1].

ALE utiliza como base **Stella**, un emulador de código abierto del hardware de Atari 2600. Stella ejecuta las ROM originales y reproduce el comportamiento de la consola; ALE añade la interfaz para agentes, la extracción de la puntuación y la detección del final de la partida. Por lo tanto, ALE no es una recreación independiente de cada juego, sino una capa experimental sobre la emulación [2].

En Space Invaders el jugador controla horizontalmente un cañón láser, dispara a una formación de invasores y evita sus proyectiles. El objetivo es destruirlos antes de que alcancen la parte inferior o eliminen al jugador. Los invasores de las seis filas del cartucho de Atari 2600 otorgan 5, 10, 15, 20, 25 y 30 puntos, y la nave de mando normalmente otorga 200 [3]. ALE transforma los puntos obtenidos durante cada llamada a step en la recompensa del entorno; por ello, la suma de recompensas del episodio representa el retorno o puntuación obtenida, salvo que posteriormente se aplique un wrapper que modifique la recompensa.

### Variantes y salto de frames

La configuración predeterminada de ALE/SpaceInvaders-v5 utiliza observaciones RGB, frameskip=4, repeat\_action\_probability=0.25 y el conjunto mínimo de acciones. Sus principales parámetros son los siguientes [4]:

- obs\_type="ram" sustituye la imagen por los 128 bytes de RAM de la consola.
- frameskip indica cuántos frames emulados se mantiene una acción antes de solicitar otra decisión. También puede ser un intervalo aleatorio.
- repeat\_action\_probability introduce *sticky actions*: con esa probabilidad se repite la acción anterior en lugar de ejecutar la nueva.
- full\_action\_space=True habilita las 18 combinaciones legales del control de Atari; el valor predeterminado expone únicamente las necesarias para el juego.

El salto de frames reduce la cantidad de decisiones, observaciones e inferencias que deben procesarse. Un salto de cuatro puede acelerar aproximadamente cuatro veces esta parte de la simulación y hace que las acciones persistan en el tiempo. Sin embargo, un valor demasiado alto puede ocultar eventos breves y disminuir el control fino. En la grabación, por ejemplo, un

proyectil puede impactar durante uno de los frames internos y no aparecer exactamente en el frame que observa el agente. Por esta razón, frameskip debe documentarse al comparar resultados.

### Espacios de observación y acción

La observación predeterminada de Space Invaders es Box(0, 255, (210, 160, 3), uint8), una imagen RGB con 100,800 intensidades. En comparación, CartPole-v1 entrega solamente cuatro valores float32: posición y velocidad del carro, y ángulo y velocidad angular del poste. Una imagen no identifica explícitamente objetos o velocidades, por lo que aumenta el consumo de memoria, suele requerir redes convolucionales y necesita varios frames para representar movimiento. CartPole ya proporciona un estado físico compacto.

La observación RAM tiene forma Box(0, 255, (128,), uint8). Puede ser útil para prototipos rápidos o para estudiar representaciones sin pagar el costo de la visión. Su limitación es que la interpretación de cada posición de memoria es específica del juego, mientras que los píxeles constituyen una entrada más general y similar a la disponible para un jugador humano [4].

El espacio mínimo de acciones es Discrete(6) [5]:

| Índice | Acción               | Efecto             |
|--------|----------------------|--------------------|
| 0      | NOOP                 | No hacer nada      |
| 1      | FIRE                 | Disparar           |
| 2 / 3  | RIGHT / LEFT         | Mover el cañón     |
| 4 / 5  | RIGHTFIRE / LEFTFIRE | Moverse y disparar |

AtariPreprocessing proporciona el preprocesamiento habitual de Atari: ejecuta acciones NOOP aleatorias al reiniciar, realiza frame skipping, aplica el máximo por píxel de los dos frames más recientes para reducir el parpadeo, convierte a escala de grises y redimensiona normalmente a 84×84. También permite normalizar los píxeles y terminar cuando se pierde una vida. El wrapper **no realiza recorte de recompensa**; para convertirla a {-1, 0, 1} se necesita un wrapper adicional. Si se usa este preprocesamiento, el entorno base debe crearse con frameskip=1 para evitar aplicar el salto dos veces [6].

FrameStackObservation mantiene las últimas  $N$  observaciones en una ventana móvil. Habitualmente se apilan cuatro frames procesados para que el agente pueda inferir dirección y velocidad, información que no está presente en una imagen aislada [7].

### Módulo desarrollado

El archivo agentes\_ale.py concentra la infraestructura en cinco funciones:

| Función       | Responsabilidad                                                                                                                                         |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| crear_entorno | Invoca gym.make; si se solicita video, configura rgb_array y aplica RecordVideo. Acepta argumentos adicionales y funciona con otros entornos Gymnasium. |

|                      |                                                                                                                                                                                                                   |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| agente_aleatorio     | Devuelve env.action_space.sample() como baseline sin entrenamiento.                                                                                                                                               |
| agente_regla_simple  | Convierte la observación a una imagen 2D, estima las posiciones horizontales del cañón y los invasores, y selecciona LEFTFIRE, RIGHTFIRE o FIRE. Si la entrada no es compatible, usa una acción aleatoria válida. |
| ejecutar_episodio    | Reinicia el entorno y aplica el agente hasta terminated, truncated o max_steps; acumula pasos, retorno y causa de finalización.                                                                                   |
| generar_video_agente | Crea el entorno grabable, ejecuta los episodios, garantiza env.close() mediante try/finally y devuelve las rutas MP4 y las métricas.                                                                              |

La consulta de get\_action\_meanings() evita asumir que los índices de acciones son iguales en todos los juegos. Asimismo, el cierre dentro de finally es indispensable porque RecordVideo termina de codificar el último MP4 durante env.close(). La detección de archivos compara tamaño y fecha de modificación, por lo que también reconoce videos sobrescritos al repetir el experimento.

Se ejecutaron cinco pruebas unitarias para comprobar acciones válidas, límite de pasos, compatibilidad con CartPole, validación de argumentos y comportamiento de la regla visual. Todas finalizaron correctamente. El episodio completo del agente aleatorio produjo **747 pasos y una recompensa total de 135.0**, y quedó guardado como entregables/videos/space-invaders-aleatorio-episode-0.mp4. Debido a que la política y las acciones pegajosas son estocásticas, estos valores pueden cambiar al repetir la ejecución.

## Conclusión

La solución separa la política del ciclo de interacción y de la grabación. Esto permite sustituir el agente aleatorio o heurístico por un modelo entrenado sin reescribir la administración del entorno, las métricas ni la generación de video. El módulo constituye así una base verificable y extensible para futuros proyectos de aprendizaje por refuerzo.

## Referencias

- [1] Bellemare, M. G. et al. (2013). [The Arcade Learning Environment: An Evaluation Platform for General Agents](#).
- [2] Farama Foundation. [Arcade Learning Environment — repositorio oficial](#).
- [3] Atari, Inc. (1980). [Space Invaders: Game Program Instructions](#).
- [4] Farama Foundation. [ALE: Environments](#).
- [5] Farama Foundation. [Space Invaders](#).
- [6] Gymnasium. [AtariPreprocessing](#).
- [7] Gymnasium. [FrameStackObservation](#).

Git: [https://github.com/SebastianUVG/Laboratorio\\_5\\_ALE](https://github.com/SebastianUVG/Laboratorio_5_ALE)